

「Dependent スコープの罠」は、標準的な CDI 実装 (Weld) と、クラウドネイティブ向けに最適化された CDI 実装 (Quarkus ArC) の間で、挙動に興味深い違いがあります。

結論から言うと、**Weld は「仕様を厳格に守り、安全側に倒す (参照を離さない)」**のに対し、

Quarkus ArC は「メモリ効率を優先し、実用的に倒す (条件付きで参照を離す)」という設計思想の差があります。

1. Weld (標準実装) の挙動：厳格な管理

JBoss EAP や WildFly で採用されている Weld では、`CDI.current().select(Foo.class).get()` で取得したインスタンスを、そのルックアップが行われたコンテキストの `CreationalContext` に紐付けます。

- **参照の保持:** 取得したインスタンスが `@Dependent` である場合、Weld はそれを「親 (呼び出し元)」が活着ている限り、**絶対に**内部リストから外しません。
- **メモリリークの理由:** 呼び出し元が `@ApplicationScoped` などの長寿命な Bean である場合、`get()` を呼ぶたびに新しいインスタンスへの参照が Weld 内部の `ArrayList` (`CreationalContextImpl.dependencies`) に蓄積されます。
- **仕様への忠実さ:** CDI 仕様では「Dependent オブジェクトは親の寿命に従う」とあるため、Weld は「親が死ぬまで勝手に GC させてはいけない (あとで `destroy` するかもしれないから)」と判断します。

2. Quarkus ArC の挙動：「不要なら捨てる」最適化

Quarkus の CDI エンジンである ArC は、パフォーマンスと低メモリ消費を重視しているため、独自の最

適化が入っています。

- **条件付きの解放:** `Instance.get()` で取得された `Dependent` オブジェクトが、**「インターセプターやデコレータを持たず、`@PreDestroy` コールバックも持たない」**単純な POJO である場合、ArC は参照を保持し続けません。
- **「掃除不要」の判断:** ArC はビルド時に Bean を解析し、「この Bean は後で明示的に破棄 (`destroy`) する必要がない」と判断できれば、`get()` して渡した瞬間に管理対象から外します。これにより、親が生き続けていても、インスタンス自体は GC の対象になります。
- **差が出るケース:** もし Bean に `@PreDestroy` が付いている場合、ArC も Weld と同様に「後でこれを実行しなければならない」ため、参照を保持し続け、同様にメモリリークのような挙動を示します。

3. 比較まとめ

特徴	Weld (Standard CDI)	Quarkus ArC
設計思想	仕様の厳格な遵守（互換性重視）	メモリ効率の最適化（実用性重視）
参照の保持	常に保持する	インスタンスの特性に応じて保持しない
メモリ解放	<code>Instance.destroy()</code> が必須	単純な Bean なら自動で GC 対象になる

特徴	Weld (Standard CDI)	Quarkus ArC

リスク	ループ内での <code>get()</code> で即座に OOM	特定の条件（インターセプター等）でリーク
-----	------------------------------------	----------------------

実務でのアドバイス

この挙動の差があるため、「**Quarkus** で動いていたから大丈夫」と思って同じコードを **WildFly/Weld** に持っていくと、**本番環境で OutOfMemory が発生する** というトラブルがよく起こります。

共通して安全な書き方は、やはり以下のように **Instance** オブジェクトを **try-with-resources** のように扱う（あるいは明示的に **destroy** する）ことです。

Java

```
Instance<MyBean> instance = CDI.current().select(MyBean.class);
```

```
MyBean bean = instance.get();
```

```
try {
```

```
    bean.doSomething();
```

```
} finally {
```

```
    instance.destroy(bean); // これで Weld でも確実に解放される
```

```
}
```

この「実装による最適化の違い」は、Java EE から Jakarta EE への移行や、クラウドネイティブ化の文脈で

非常に重要な視点です。